Web Engineering & Development (SWE 363)

Back-End Development: Node.js & Express II

In today's lecture:

- Three ways to send data: query string, form body, route parameter
- Route parameter middleware
- Sending JSON responses

Reference:

- Zybook: 6.5

Activity: Warm-up from last lecture (2 min)

```
const logRequest = function(req, res, next) {
  console.log(`Request: ${req.method} for ${req.path}`);
  next();
};
app.use(logRequest);
app.use(express.static("public"));
app.post("/goodbye", function(req, res) {
  res.send("<h1>Goodbye, Express!</h1>");
});
```

1. Which route matches a GET request to `/goodbye`?
2. What do the parameters `req`, `res`, and `next` represent?
3. If `next()` is removed, what is logged for `GET /goodbye`?
4. To be served, must `hello.html` be in `public` or in `node_modules`?

6.5 Express Request Data

Three Ways to Send Data

> The client sends data in the **query string**, the request **body**, or the URL **path**

Express puts each one in its own object on `req`, ready for the route to use

Data	Example request	Read it from
Query string	`GET /hello?name=Bob`	`req.query.name`
Form POST body	`POST /hello` (body `name=Tamika`)	`req.body.name`
Route parameter	`GET /users/jblack`	`req.params.username`

Query String Parameters

> Express **automatically parses** the `name=value` pairs after the `?` into **`req.query`**

The same route can greet anyone: only the URL changes

```
// GET http://localhost:3000/hello?name=Bob&age=21
app.get("/hello", function(req, res) {
  const html = `<h1>Hello, ${req.query.name}!</h1>
    <p>You are ${req.query.age} years old.</p>`;
  res.send(html);
});
```

Browser shows: Hello, Bob! You are 21 years old.

Activity: Predict what the page shows

```
app.get("/hello", function(req, res) {
  const html = `<h1>Hello, ${req.query.name}!</h1>
    <p>You are ${req.query.age} years old.</p>`;
  res.send(html);
});
```

1. `/hello?name=Sara&age=20`
2. `/hello?age=20`
3. `/hello?name=Doctor+Who&age=900` (in a URL, `+` encodes a space)
4. `/hello?city=Dallas` : is the city in `res.query.city` or `req.query.city`?

Posting Form Data

> A form with `method="post"` sends its data **URL-encoded** in the **body** of the HTTP request

The data reaches the server without appearing in the URL

```
<!-- public/hello.html -->
<form method="post" action="/hello">
  <label>Name? <input type="text" name="name"></label>
  <label>Age? <input type="number" name="age"></label>
  <input type="submit" value="Submit">
</form>
```

Body sent on Submit: `name=Tamika&age=24`

Reading Form Data

> `express.urlencoded()` is **middleware** that parses the URL-encoded body and adds the values to **`req.body`**

Enable it with `app.use()` before the route that reads `req.body`

```
app.use(express.static("public"));                // serves hello.html
app.use(express.urlencoded({extended: false}));  // body → req.body

app.post("/hello", function(req, res) {
  const html = `<h1>Hello, ${req.body.name}!</h1>
    <p>You are ${req.body.age} years old.</p>`;
  res.send(html);
});
```

Activity: What happens if… (each change on its own)

```html
<form method="post" action="/hello">
  <input type="text" name="name">
  <input type="submit" value="Submit">
</form>
```

```
app.use(express.static("public"));
app.use(express.urlencoded({extended: false}));
app.post("/hello", function(req, res) {
  res.send("Hello, " + req.body.name + "!");
});
```

1. Submit is clicked with no name?
2. `express.urlencoded()` is removed?
3. The form uses `method="get"` ?
4. The body is sent as `application/json` ?
5. The `express.static()` line is removed?

Route Parameters

> A **route parameter** is a value in the URL path, named with a **colon** (`:username`);
> Express attaches it to `req.params`

One route makes a dynamic webpage for every username

```
// GET http://localhost:3000/users/jblack
app.get("/users/:username", function(req, res) {
  const username = req.params.username;   // jblack
  res.send("<h1>Profile for " + username + "</h1>");
});
```

Browser shows: Profile for jblack

Activity:

Which req property (`req.query` , `req.params` , or `req.body`)? What value?

1. Route `/users/:username` , request `GET /users/abc`
2. Route `/search` , request `GET /search?term=node`
3. Route `app.post("/major", ...)` , a form posts `major=SWE`
4. Route `/city/:cityName` , `GET /city/Denver` : is it `req.params.city` ?
5. Route `/users/:username` , request `GET /users/` : does the page say "Profile for "?

Route Parameter Middleware

> `app.param()` defines middleware that runs **before** the route's callback; its **fourth parameter** is the parameter's value

It prepares data (`req.user`) for the route, then **must call** **`next()`**

```
app.param("username", function(req, res, next, username) {
  if (username === "jblack") { req.user = { name: "Jack Black" }; }
  else { req.user = { name: "Unknown" }; }
  next();   // continue processing the request
});

app.get("/users/:username", function(req, res) {
  res.send("<h1>Profile for " + req.user.name + "</h1>");
});
```

Sending JSON Responses

> `res.json()` encodes a JavaScript object as **JSON**; `res.status(404)` sets the **status code** before it

A **web API** sends data (JSON) instead of an HTML page

```
app.get("/users/:username", function(req, res) {
  if (req.user.name !== "Unknown") {
    res.json(req.user);                              // {"name":"Jack Black"}
  }
  else {
    res.status(404).json({ error: "Unknown user" });  // 404 = Not Found
  }
});
```

Activity: Predict the response

```
app.param("zip", function(req, res, next, zip) {
  if (zip === "80015") {
    req.forecast = {high: 95, low: 72};
  }
  next();
});
app.get("/weather/:zip", function(req, res) {
  if (req.forecast) { res.json(req.forecast); }
  else { res.status(404).json({ error: "Unknown ZIP code" }); }
});
```

1. `/weather/80015`
2. `/weather/12345` : what status code?
3. `next()` is removed: `/weather/80015` ?
4. `app.param("zipcode", ...)` instead of `"zip"` : `/weather/80015` ?

Activity: Exit ticket: find the 4 bugs

```
app.param("courseId", function(req, res, next, courseId) {
  req.course = { code: courseId };
});
app.get("/courses/:code", function(req, res) { res.json(req.course); });
// GET /search?term=node
app.get("/search", function(req, res) { res.send(req.params.term); });
// A form posts major=SWE (no other app.use() calls)
app.post("/major", function(req, res) { res.send(req.body.major); });
```

Key Takeaways

- **Query string** (`?name=Bob&age=21`) → `req.query`
- **Form POST body** → `express.urlencoded()` parses it into `req.body`
- **Route parameter** (`/users/:username`) → `req.params`
- `app.param()` runs **before** the route's callback and must call `next()`
- `res.json()` sends JSON; `res.status(404).json()` also sets the status code

30m

Demo

6.4 Request Data in Express

- Query string parameters
- Route parameters
- Param middleware
- JSON responses with status codes

Next Class

- Mongoose (7.1)
- MongoDB (7.2)